

Extending TRIE for Finding Invariants in Rust Crates

Background

Ensuring correctness in systems software often depends on invariants: conditions that must always remain true when code is executed, across module boundaries, API calls, and state transitions. For example, a memory allocator must guarantee that every block of memory is freed exactly once, or a network driver must ensure that device registers are written only when the hardware is in the correct state. Invariants like these are critical to preventing crashes, data corruption, and security vulnerabilities. Rust provides a foundation for enforcing certain invariants through its affine type system, which is built around ownership, borrowing, and lifetimes. By restricting aliasing and guaranteeing that resources have a single owner, the compiler can catch many classes of bugs at compile time.

In parallel, recent work in large language models (LLMs) has explored multi-agent systems designed for repository-level tasks. These systems can navigate large projects, search for definitions and references, and retrieve context such as documentation or comments that explain developer intent. With autonomous tool use—search, parsing, and call-graph exploration—LLM agents can reason over both code structure and natural language artifacts [2][3]. This project brings these two directions together. This project will extend the TRIE (Textual Retrieval & Invariant Extraction) system to Rust, enabling it to automatically find invariants in real-world Rust repositories.

Goal and Objectives

Extend TRIE to Rust Codebases

Overall Goal. The aim of this project is to build a research prototype that can automatically discover invariants in Rust crates. By extending TRIE (Textual Retrieval & Invariant Extraction) to Rust, the system will be able to navigate codebases, extract both implicit invariants, and store them in a structured schema.

Rust Crate Navigation: Following recent work on agentic code navigation, each Rust crate will be represented as a structured graph that combines both structural and textual signals: type definitions, call and symbol graphs, documentation comments, module layout, and Cargo metadata. Instead of exposing the entire repository at once, the agent will traverse this graph selectively, guided by retrieval strategies similar to Monte Carlo Tree Search and ReAct-style reasoning. Tools such as symbol lookup, call-graph expansion, and cross-reference search allow the agent to move between definitions, callers, and usage sites as needed. This design makes it possible to navigate large repositories efficiently and focus only on the code paths most relevant to a candidate invariant.

Invariant Mining. The second objective is to mine invariants from the indexed code.

- *Invariants* will be inferred from patterns in the code that Rust's type system can already enforce. Example invariants include:
 - **Uniqueness.** An invariant ensuring that only one valid representation of a resource exists at a time. Enforced when a type cannot implement `Clone` or `Copy`, has private fields, and transfers ownership by move rather than duplication.
 - **Pairing.** An invariant ensuring that resource acquisition is always matched with release. Commonly enforced when a type implements `Drop`, so allocations or locks are automatically cleaned up at scope end, but it can also be enforced through consuming APIs or tpestate transitions that require explicit closure.
 - **Temporal Order.** An invariant ensuring that operations occur in a legal sequence. Enforced when methods consume a value of one type and return another, as in tpestates, so the compiler disallows calling methods out of order.
 - **State Consistency.** An invariant ensuring that resources are only used in states where operations are valid. Enforced when different states are encoded as distinct types (e.g., `Enabled` vs. `Disabled`), so the compiler exposes only the methods appropriate for each state.

Representation. To make these invariants easily understandable and searchable by the developer, they will be stored in a standard schema with six fields:

- **Subject:** the specific code entity (type, function, or module) the invariant applies to.
- **Resource:** the underlying system object being represented (e.g., memory page, network queue).
- **State:** the condition or mode of the resource (e.g., Allocated, Free, Enabled, Disabled).
- **Operation:** the action being restricted or required (e.g., `Clone`, `receive()`, write to register).
- **Guard:** the condition under which the operation is legal (e.g., "only if Enabled," "never allowed to clone").
- **Obligation:** the follow-up requirement to maintain correctness (e.g., "must free after allocate," "must close after open").

Evaluation.

The extended TRIE system will be evaluated along two axes:

1. **Correctness.** Compare mined invariants against Rust's compiler-enforced guarantees and human-written documentation. For a subset of crates, manually validate whether extracted invariants align with intended usage.
2. **Scalability.** Compare accuracy on crates of different sizes to evaluate mining strategy precision on both large and small repositories.

Research Question

- **RQ (Discovery):** How accurately can the agent mine invariants from real repositories?

Deliverables and Timeline

Deliverables

- Open-source repository and documentation

Timeline (tentative)

Phase 0 — Setup (Sept 18-Sep 29)

- Pick repos to study and review related work.

Phase 1 — Indexing (Sept 29- Oct 13)

- Extract AST/symbol graph, gather docs and comments, set up code search tools.

Phase 2 — Invariant Mining (Oct 13- Nov 2)

- Implement invariant mining from rust crates and create schema.

Phase 3 — Validation I (Nov 2 - Nov 17)

- Annotate a gold set of invariants, measure accuracy.

Phase 4 — Reporting (Nov 17- Dec 1, Dec 1-Dec 11)

- Final writeup, presentation.

References

[1] Ijaz, Ramla, et al. 'Leveraging Rust for Lightweight OS Correctness'. Proceedings of the 1st Workshop on Kernel Isolation, Safety and Verification, Association for Computing Machinery, 2023, pp. 1–8, <https://doi.org/10.1145/3625275.3625398>. KISV '23.

[2] Eniser, Hasan Ferit, et al. "Towards translating real-world code with llms: A study of translating to rust." *arXiv preprint arXiv:2405.11514* (2024).

[3] Liu, Xiangyan, et al. "Codexgraph: Bridging large language models and code repositories via code graph databases." *arXiv preprint arXiv:2408.03910* (2024).